EYESEEYOU

Eye See You

A system to detect the presence of obstacles for visually-impaired users, using text to speech and vibrations as a guidance mechanism

Presented by:

Cecchini Leonardo

Protopapa Matteo

Ruisi Rosario

Architecture

- ARCore
- Voice synthesis (TTS)
- Tactile feedback via vibration
- Smartwatch
- Voice Control

ARCore

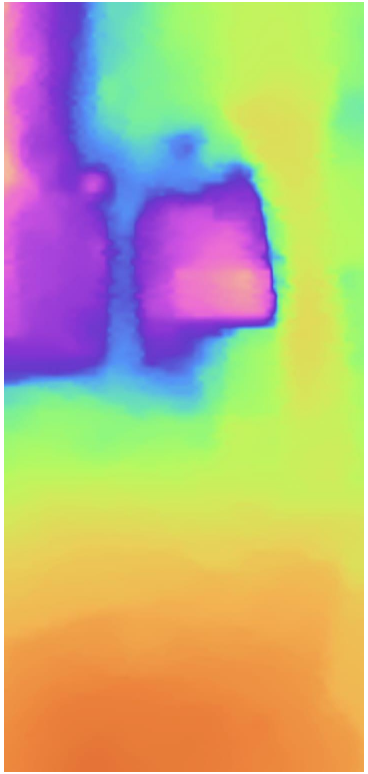
Google's Development Kit for AR Applications

ARCore offers:

- Depth perception
- Semantic understanding
- Light estimation
- Image stabilization
- Anchor placement

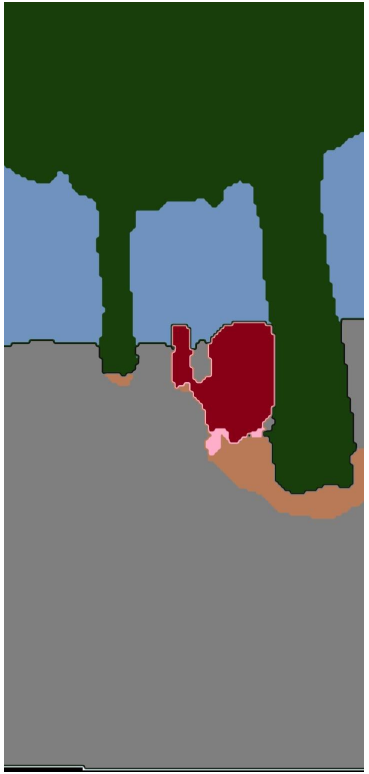
Depth Image

Uses images from different angles to estimate object distances from the camera, assigning a depth value to each pixel.



Semantic Image

Applies machine learning algorithms to classify each pixel by assigning a label based on the detected object type.

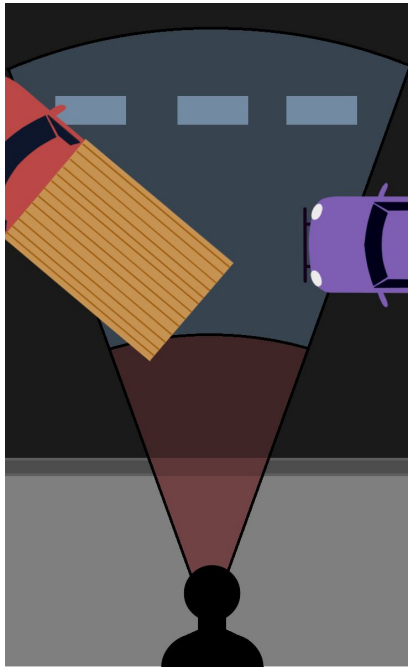


Obstacle Detection 🚧

The obstacle detection process is divided in four steps:

1. Uses previously acquired images to **create a map**, assigning each label to its corresponding pixels with a distance less than 3.5 meters.

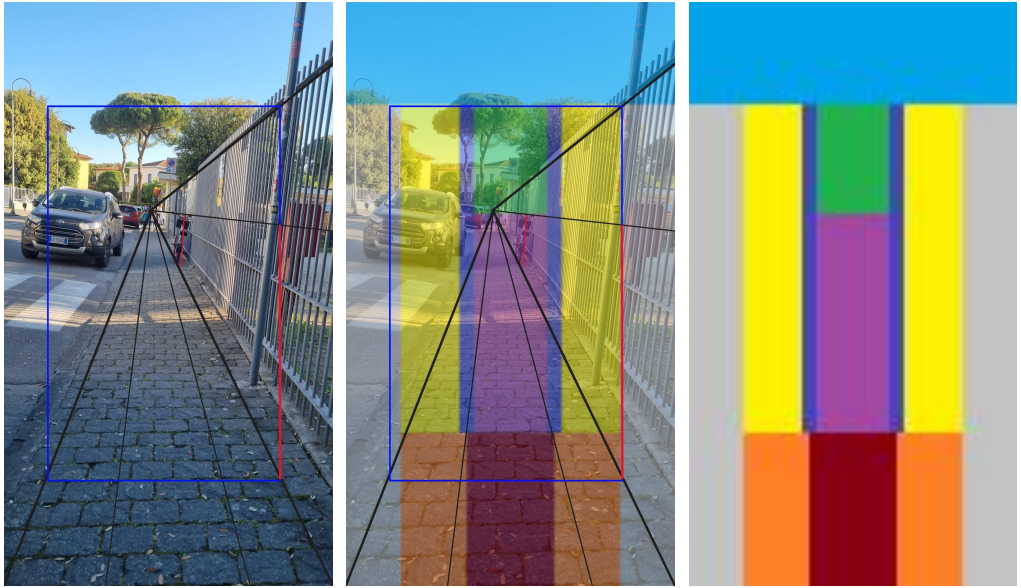
2. *Identifies nearby labels* with points closer than 2 meters.



Zones Identification

1. **Assigns each label** to a set of zones where it is densely distributed.

2. **Compares the results** from three different frames before computing the final output.



Step Detection

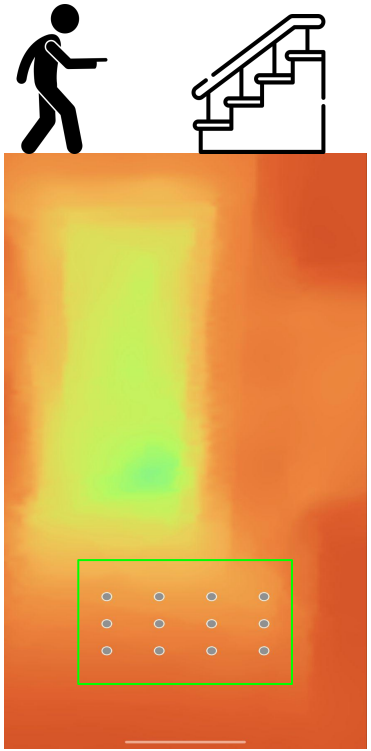
Two Main Features:

➔ **recognize** a valid step

➔ **warn** the user when he is very close to a step

How

Using a 3x4 point grid situated on the lower part of the screen (green rectangle in the image)



Almost On A Step

1. When we detect a step we need to **store** its **Pose** to retrieve the real-world coordinates

2. Check if we are almost on a step using **current** camera **Pose** to warn the user

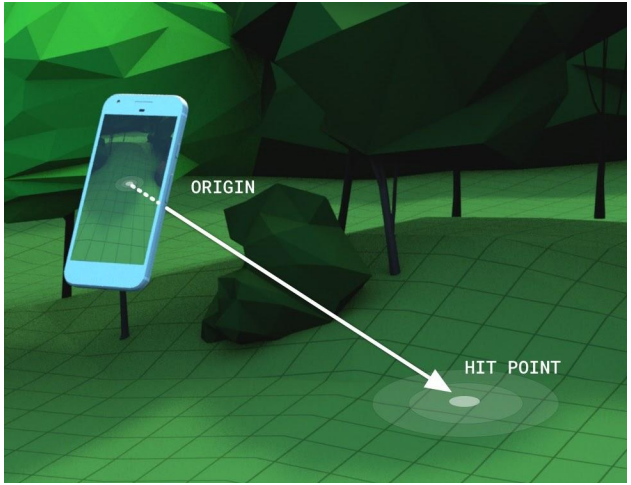
ARCore Pose *represents the 3D position and orientation of an object in space. Contains a transformation matrix, position (x, y, z), and rotation (quaternion)*

Recognize A Step

1. Conversion of screen points into **real-world points** (hit test)

2. Computing **height** and **depth difference** between different rows

3. Check if a new step detected must be **stored** (if camera has rotated or does not belong to previous stair)



Hit test working example

Voice Feedback & Voice Commands

- Text-to-Speech engine for spoken alerts
- Voice commands (no need to touch the screen)
- Recognized actions
- Optional touchscreen control
- Auto-restart if recognition fails

Vibration Feedback

- Vibration motor for tactile feedback
- Toggle vibration on/off
- Optional Smartwatch support

Results & Analysis

SETUP:

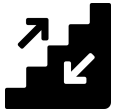
- Android devices (Samsung Galaxy s21, OPPO Find N2 flip)
- Smartwatch with Wear OS (TicWatch)

ENVIRONMENTS:

- Indoor and outdoor
- Static and dynamic obstacles
- Near stairs
- Different lighting conditions

Results & Analysis

Obstacle detection: satisfactory accuracy especially outdoors and in wider areas, while many interferences in tighter zones



Step detection: Acceptable performances in a range of 1 meter (closer to the user)

Lighting condition impact: Under optimal lighting, consistent and accurate depth tracking, in low-light environments increased errors and reduced obstacle recognition

Limitations

Low refresh rate: due to the *complexity of the operations* performed, comports a *reduction* on the *precision* of the semantic and depth image

Highly dependent on light conditions: ARCore struggles in low-light environments, as its motion tracking and environmental understanding rely heavily on *visual features*, which are *harder to detect* in poor lighting conditions.

Future Work

EYESEEYOU

- **Optimization** and reduce resource consumption
- Dynamically **detect the type of environment** and adapt the app settings
- Dynamically **adapt the message rate** based on user's speed
- **Enhance obstacle and step recognition** by integrating advanced machine learning-based object detection models like YOLOv5 or MediaPipe

MediaPipe

YOLOv5

Lambda

EYESEEYOU

Thanks for your Attention!